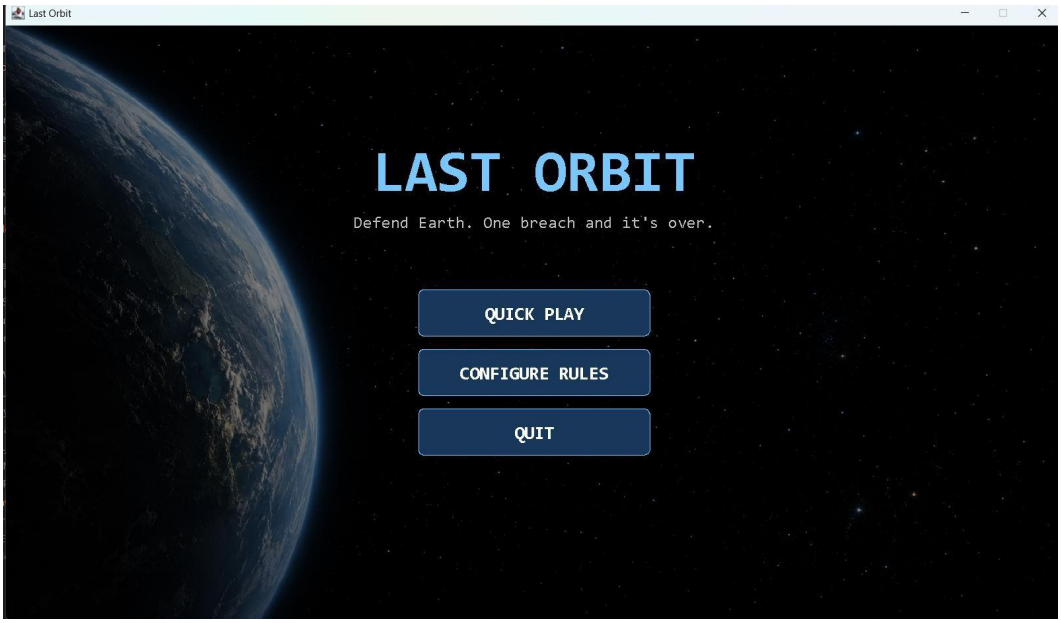
OBJECT ORIENTED PROGRAMING

PROJECT REPORT(LAST ORBIT GAME)



Name - Robel Teshome Eshete

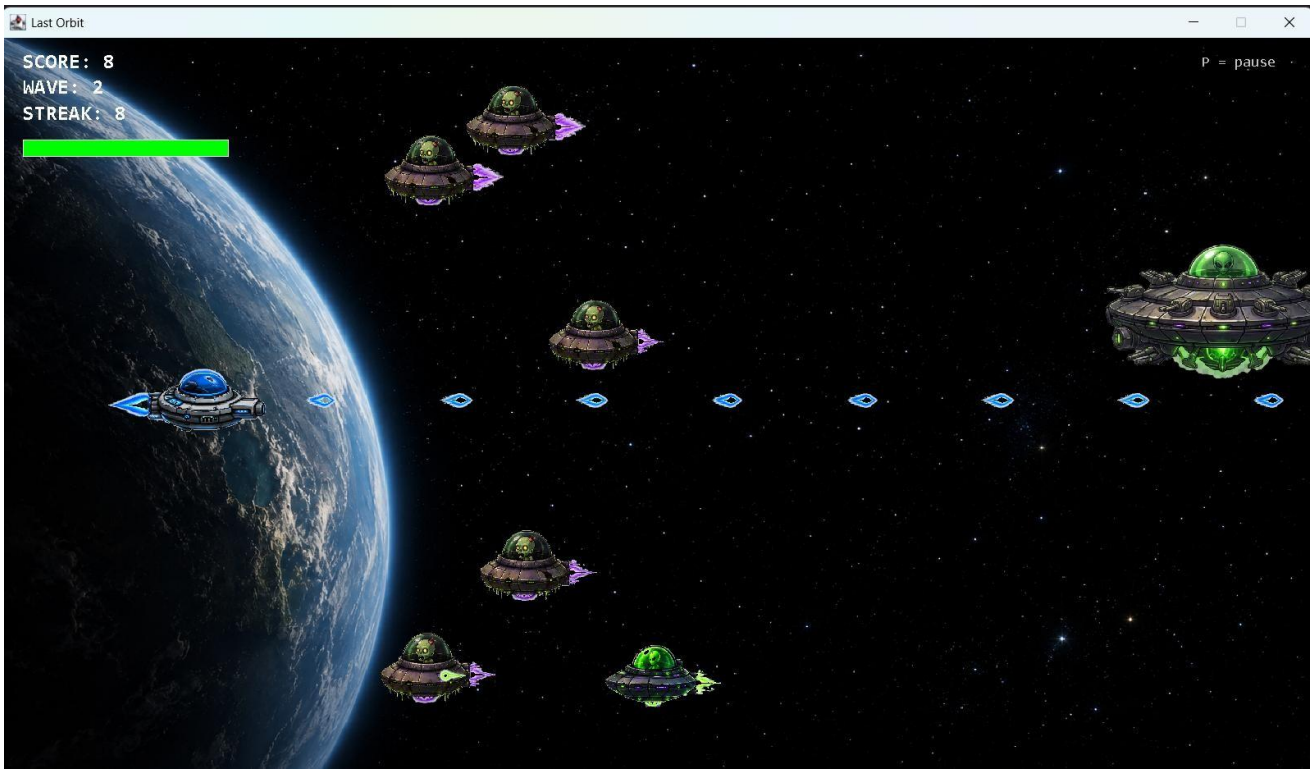
Matricola - 573565

Submitted to- Prof. Distefano **Date - 02/07/2026**

Introduction

Last Orbit is a 2D arcade space-shooter in which the player defends Earth from waves of alien UFOs. It is written in **pure Java SE 17** with a **Java Swing / Graphics2D** frontend, using only the standard JDK. Every concern — game state, rendering, input, collision, rules, file storage, and error reporting — is modelled in plain object-oriented terms.

This report walks through each OOP concept, using code drawn directly from the project, and follows the distinction between the **creator programmer**, who writes the implementation, and the **user programmer**, who works against the interface.



the running game, showing the player ship, incoming aliens, and the HUD (score, wave, health).

System Overview

The game is organised into packages, each with a single responsibility: `core` (window, game loop, input), `entities` (player ship, aliens, projectiles, power-ups), `components` (weapon, shield), `interfaces` (the capability types), `combat` (collision resolution), `waves` (wave generation, boss, adaptive difficulty), `upgrades` (timed power-ups), `rules` (the customizable rule engine), `gui` (menu and screens), `score` (high-score file I/O), `audio` (music), and `exceptions` (the custom exception hierarchy).

An abstract `Entity` is the root of every on-screen object. A generic `EntityManager<T>` updates and draws collections of them polymorphically. A `GameLoop` drives the whole game on its own thread at a fixed frame

rate, and a `ScreenManager` switches between the menu, the rule-configuration screen, and the game.

OOP Concepts Used

The rest of the report covers each OOP concept in its own section, with real code from the project, why the decision was made, and the creator-programmer / user-programmer perspectives.

1. Information Hiding

Information hiding separates the **private part (the implementation)** from the **public part (the interface).** so that other classes depend only on what they need and never on details that might change. I**nformation hiding and encapsulation are orthogonal**, independent ideas that happen to reinforce each other.

How it works

Every entity keeps its state private and exposes it only through methods. The `Entity` base class hides its position and size completely; nothing outside can move an entity directly, and callers see only public accessors such as `getX()`, `getBounds()`, and `isAlive()`. The mutators that actually change position are `protected`, so only subclasses inside the hierarchy may use them.

```
public abstract class Entity {
    private final Vector2D position;
    private final int width;
    private final int height;
    private boolean alive;
```

the Entity.java private field

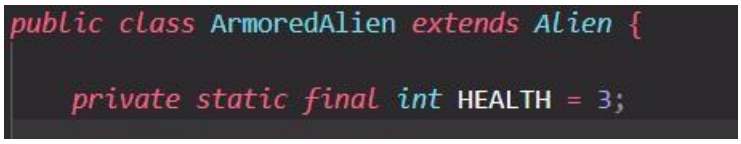
Information-Hiding Engineering — three steps for every attribute

The screenshot below shows the `Entity` constructor and its getters. It is a compact illustration of the three decisions taken for every attribute in the project.

```
public Entity(double x, double y, int width, int height) {
    this.position = new Vector2D(x, y);
    this.width = width;
    this.height = height;
    this.alive = true;
}
public double getX() { return position.getX(); }
public double getY() { return position.getY(); }
public int getWidth() { return width; }
public int getHeight() { return height; }
public boolean isAlive() { return alive; }
```

Step 1 — make the attribute private. Every field in the class is declared `private`. In the screenshot, none of `position`, `width`, `height`, or `alive` is reachable from outside; the only way in is through the getters. This holds across the whole project — every field of `Alien`, `Spaceship`, `Weapon`, and the rest is private, and other classes reach state only through methods that keep the object consistent.

Step 2 — decide whether the attribute is an instance attribute or a class attribute. An **instance** attribute has one independent copy per object; a **class** attribute (declared `static`) has a single copy shared by all objects of the type. In `Entity`, `position`, `width`, `height`, and `alive` are all instance attributes, because every entity must have its own position, size, and alive-state. By contrast, fixed values that are the same for every object of a type are made class attributes — for example `BOSS_HEALTH` and `SPAWN_INTERVAL` in `BossAlien`, where one shared copy is correct.



Step 3 — decide whether the attribute is constant or variable, and initialize it in the constructor. A value that must never change after the object is built is declared `final`. In the screenshot, `width` and `height` are `final` constants (a sprite's size never changes), while `alive` is a variable that flips to `false` when the object is destroyed. Two consequences follow, both visible in the code:

- **Constants are assigned in the constructor.** A `final` field cannot be left unassigned. When an object is created, the superclass `Object` constructor runs first and default-initializes fields — numeric fields become `0`, `boolean` fields become `false`, and reference fields (such as `String` or any object) become `null`. Those defaults are almost never the values we want, so the `Entity` constructor explicitly assigns `width` and `height` from its parameters. If a `final` field were left unassigned, the compiler would reject the class outright.
- **Constants need no setter.** Because a `final` value can never change after construction, it is exposed with a getter only — there is no `setWidth()` or `setHeight()`. Only genuinely variable state (like the alive-flag, through `setAlive()`) gets a mutator.

The same three decisions recur everywhere. In `Alien`, `maxHealth` and `speed` are `private final` constants assigned once in the constructor and never given setters, while `health` is a `private` variable that changes as the alien takes damage. In `WaveParameters`, every field is `private final`, making the whole object immutable once built — it carries only getters, no setters at all. This consistency is what makes the objects safe to pass around: a value marked constant genuinely cannot be altered by any other class.

Why I did this

A game touches shared state every frame. Hiding each object's fields behind a small, deliberate set of methods means state can change only in controlled, correct ways — a bug can never come from an outside class quietly overwriting a coordinate or a health value.

From the Programmer's Perspective (creator). When writing Entity, you make each field private, decide instance-versus-class, and decide constant-versus-variable, assigning every constant in the constructor. Each decision is a commitment: callers cannot depend on what you hide, so you remain free to change the implementation later.

From the User's Perspective. A class using an Entity never sees its position field, and finds no setter for its size. It calls getBounds() or getX() and trusts the object to stay consistent. The small surface area means fewer ways to misuse the object.

2. Encapsulation

Encapsulation bundles an object's **state together with the behavior** that acts on it, so the object can never be placed into an invalid state. It is independent of information hiding: hiding is about privacy, encapsulation is about keeping state and its operations together.

How it works

The `Shield` component bundles its remaining-hits count with the only operation allowed to change it, so the count can never go negative or be edited from outside. `Spaceship` does the same with health: callers cannot subtract from health directly; they call `takeHit(damage)`, and the ship itself decides whether a shield absorbs the hit, whether an invulnerability window applies, and when it dies.

```java
package lastorbit.interfaces;

public interface Destructible {
    void takeHit(int damage);
    boolean isDestroyed();
}
```

```java
@Override
public void takeHit(int damage) {
    if (shield != null && shield.absorbHit()) {
        return;
    }
    if (invulnTimer > 0) {
        return;
    }
    health -= damage;
    if (health < 0) {
        health = 0;
    }
    invulnTimer = INVULN_FRAMES;
    if (health <= 0) {
        setAlive(alive: false);
    }
}
```

the Spaceship.java takeHit(int damage) method, showing the shield-absorb check

Rationale

The object is the boundary where the rules live. Because every hit flows through `takeHit`, the ordering — shield first, then invulnerability, then damage — is enforced in exactly one place and cannot be bypassed.

From the Programmer's Perspective. You bundle the health field with takeHit so the two can never drift apart. The method is the only legal entry point into the object's state.

From the User's Perspective. The collision code calls player.takeHit(damage) and the ship reacts correctly on its own. The caller only detects the hit; correctness of the health state is the ship's job.

3. Inheritance

Inheritance models an **"is-a"** relationship and is concerned with the **implementation part** (the creator programmer). It is a **white-box** form of reuse, because the subclass inherits the internals of the superclass. It is also called **specialization** (general→specific).

How it works

`Alien` is an abstract base holding everything common to enemies — health, speed, movement toward Earth, the freeze flag, and the hit logic. Concrete enemies specialise it: `StandardAlien` (one hit), `ArmoredAlien` (three hits, degrades through damage states), `MiniAlien` (spawned by the boss), and `BossAlien` (high health, patrol movement). Each reuses all of `Alien`'s behavior and adds only what makes it different, so a new enemy type costs almost nothing.

```
public abstract class Entity {
```

```
public abstract class Alien extends Entity implements Movable, Destructible, Shooter {
```

```
public class BossAlien extends Alien implements MiniSpawner {
```

```
public class StandardAlien extends Alien {
```

```
public class ArmoredAlien extends Alien {
```

```
public class MiniAlien extends Alien {
```

The entity class hierarchy

Key design decision — the abstract constructor

`Alien` is abstract and cannot be instantiated, yet it has a constructor, and this is required: every subclass constructor calls `super(...)`, and without a matching constructor in the abstract base that call would not compile. `Alien`'s constructor also never calls an overridable method, because doing so would dynamically dispatch to a subclass method before the subclass is fully constructed, producing a subtle logical error.

Why I did this

Inheritance must encode genuinely shared behavior. Movement, taking damage, and reaching Earth are shared by every enemy, so they belong in `Alien`. What differs — health, appearance, whether it shoots — is left to the subclasses.

From the Programmer's Perspective. You ask what is true of every alien and put it in the abstract base. Declaring abstract Animation activeAnimation() forces every future subclass to answer “what do you look like right now?”; the compiler rejects any subclass that omits it.

From the User's Perspective. The game loop holds an Alien reference and calls update() and render() without knowing which concrete enemy it is.

4. Abstraction

Abstraction expresses **what** something does without committing to **how**. It is the reverse direction of specialization (specific→general), and it is what makes it possible to enforce the Open/Closed Principle. Last Orbit uses abstraction at two levels: abstract classes and small, single-method interfaces.

Abstract classes

`Entity`, `Alien`, `Projectile`, `Weapon`, `Upgrade`, and `Wave` are all abstract. Each defines a concept generically and leaves the specifics to subclasses. `Wave`, for instance, knows how to spawn a roster over time but not *what* to spawn — that is left to `StandardWave` and `BossWave`.

```
public abstract class Wave {

    private final WaveParameters params;
    private final EnemyFactory factory;
    private final int screenWidth;
    private final int screenHeight;
    private final Random random;

    private final int totalToSpawn;
    private int spawnedCount;
    private int spawnTimer;
```

On the same class we have

```
public void update(EntityManager<Alien> aliens) {
    if (isFinishedSpawning()) {
        return;
    }
    spawnTimer++;
    if (spawnTimer >= params.getSpawnSpacing()) {
        spawnTimer = 0;
        aliens.add(spawnOne());
        spawnedCount++;
    }
}
```

Single-method interfaces

The capability types are deliberately tiny and focused — it is good practice to define a single method per interface. `Movable` declares `moveStep()`; `Destructible` declares `takeHit()` and `isDestroyed()`; `Damaging` declares `getDamage()` and `onHit()`. Code that consumes them depends on the small interface, not on any concrete class.

```
package lastorbit.interfaces;

public interface Movable {
    void moveStep();
}
```

```
package lastorbit.interfaces;

public interface Destructible {
    void takeHit(int damage);
    boolean isDestroyed();
}
```

```java
package lastorbit.interfaces;

public interface Damaging {
    int getDamage();
    void onHit();
}
```

Some examples of the interfaces package: Movable.java, Destructible.java, and Damaging.java

From the Programmer's Perspective. You publish a small interface such as Destructible and implement it in several classes; the code that consumes it only ever sees the interface.

From the User's Perspective. Working against Destructible, you can call takeHit() on anything destructible — an alien, a boss, or the player — without knowing the concrete type.

5. Composition

Composition is a **"has-a"** relationship and a **black-box** form of reuse: a class is used without any dependence on its internals — the opposite of inheritance's white box.

How it works — the composition showpiece

The `Spaceship` **has-a** `Weapon` and **has-a** `Shield`, holding them as parts it delegates to rather than inheriting from. The strongest demonstration is that the weapon is **swapped at runtime** when the Enhanced-Weapon upgrade fires: the ship replaces its `StandardWeapon` with a `SpreadWeapon` and swaps it back when the upgrade expires. The firing code never changes — it simply calls `weapon.fire(...)`, and whichever component is currently plugged in responds. The `Shield` is attached and detached on the live ship in the same way.

```java
public class Spaceship extends Entity implements Destructible {

    private Weapon weapon;
    private Shield shield;
```

```java
public void setSpreadEnabled(boolean enabled) {
    if (enabled == spreadEnabled) {
        return;
    }
    spreadEnabled = enabled;
    try {
        this.weapon = enabled
            ? new SpreadWeapon(screenWidth)
            : new StandardWeapon(screenWidth);
    } catch (AssetLoadException e) {
        spreadEnabled = !enabled;
    }
}
```

the Spaceship.java weapon plus the setSpreadEnabled(...) method that replaces the weapon component at runtime

Inheritance versus composition

The project uses both deliberately: `Spaceship extends Entity` is inheritance (white box) — a ship genuinely *is-an* entity and reuses its internals — while `Spaceship` *has-a* `Weapon` is composition (black box) — the ship uses the weapon without knowing its internals and can replace it. Choosing the right one per relationship is the white-box / black-box distinction in practice.

Why I did this

Weapon and shield behavior change independently of the ship. Making them separate composed components means a new weapon type can be added without touching `Spaceship` at all.

6. Polymorphism

Polymorphism is the same definition applied in different ways, and it is most relevant to the user programmer. It divides into **ad-hoc** (coercion, overloading) and **universal** (parametric, inclusion). All are present; inclusion polymorphism is the most important and is covered last.

6a. Ad-hoc — Overloading (static binding)

Overloading gives the same method name several parameter lists, and the compiler chooses which to call from the static argument types (static binding). `Weapon` overloads `fire`: `fire(x, y)` for a single shot and `fire(spreadCount, x, y)` for a fan of shots. A second entity-side example is `Vector2D`, whose `add` operation is overloaded to accept either two separate coordinates or another vector — the same conceptual operation offered in more than one convenient form.

```
public PlayerProjectile fire(double x, double y) {
    Animation anim = new Animation(boltFrames, fps: 12);
    return new PlayerProjectile(x, y, width: 34, height: 16, anim, screenWidth);
}

public abstract List<PlayerProjectile> fire(int spreadCount, double x, double y);
```

This is a method inside the Weapon class

```
@Override
public List<PlayerProjectile> fire(int spreadCount, double x, double y) {
    List<PlayerProjectile> shots = new ArrayList<>();
    shots.add(fire(x, y));
    return shots;
}
```

And this is inside the StadardWeapon class, which is a subclass of the Weapon class and the second method is overridden inside the sub class

6b. Ad-hoc — Coercion

Coercion is the automatic conversion of a value from one type to another. It appears in the entity code where an integer health value is divided to compute a `double` health ratio: the `int` is promoted to `double` so the division is well defined, and that ratio then selects which damaged sprite an armored alien shows. Java performs the widening from the narrower type to the wider one automatically.



```
@Override
protected Animation activeAnimation() {
    double ratio = getHealthRatio();
    int index = (int) ((1.0 - ratio) * states.length);
    if (index < 0) { index = 0; }
    if (index >= states.length) { index = states.length - 1; }
    return states[index];
}
```

the ArmoredAlien.java activeAnimation() method (or Alien.java getHealthRatio()), showing the int health promoted to double in the division.

6c. Universal — Parametric Polymorphism (Generics)

Parametric polymorphism makes the *type* itself a parameter. Java implements it with **static type checking**: the type argument is fixed and verified by the compiler before the program runs. This means a mistake — such as putting the wrong kind of object into a typed collection — is caught at compile time, and cannot slip through to become a run-time crash. (The alternative, dynamic type checking used in some languages, only detects such a mistake while the program is running.) `EntityManager<T extends Entity>` is a single generic collection reused for every entity type. The bound `<T extends Entity>` is what lets the manager call `update()`, `render()`, and `isAlive()` on its contents, because the compiler knows every `T` is at least an `Entity`. One class serves projectiles, aliens, and power-ups with no per-type code, and the compiler guarantees a manager declared for one type can never hold another.

```java
public class EntityManager<T extends Entity> {

    private final List<T> entities;

    public EntityManager() {
        this.entities = new ArrayList<>();
    }

    public void add(T entity) {
        entities.add(entity);
    }

    public void updateAll() {
        Iterator<T> it = entities.iterator();
        while (it.hasNext()) {
            T entity = it.next();
            entity.update();
            if (!entity.isAlive()) {
                it.remove();
            }
        }
    }

    public void renderAll(Graphics2D g) {
        for (T entity : entities) {
            entity.render(g);
        }
    }
```

the EntityManager.java class in full

6d. Universal — Polymorphism by inclusion (dynamic binding)

This is the heart of OOP: a supertype reference can hold any concrete subtype, and method calls are dispatched at run time to the correct override (dynamic binding). The project relies on it throughout.

The game loop drives every entity through base references

`EntityManager` calls `update()` and `render()` on an `Entity` reference. At run time this dispatches to `StandardAlien`, `BossAlien`, `PlayerProjectile`, and the rest — the loop never asks which type it holds.

Collision resolved through interface types, avoiding RTTI

Collisions never inspect an object's concrete type. A projectile is `Damaging` and an alien is `Destructible`, and a hit is resolved purely through those types: the code calls `alien.takeHit(bolt.getDamage())` and `bolt.onHit()`. An `ArmoredAlien` and a `StandardAlien` are damaged by the identical call; the different outcome is decided by dynamic binding inside `takeHit`, not by any type test in the collision code.

```java
public final class CollisionManager {

    private CollisionManager() { }

    public static int resolve(EntityManager<PlayerProjectile> projectiles,
                              EntityManager<Alien> aliens) {
        int destroyed = 0;

        for (PlayerProjectile bolt : projectiles.getEntities()) {
            if (!bolt.isAlive()) {
                continue;
            }
            for (Alien alien : aliens.getEntities()) {
                if (!alien.isAlive()) {
                    continue;
                }
                if (bolt.getBounds().intersects(alien.getBounds())) {
                    alien.takeHit(bolt.getDamage());
                    bolt.onHit();
                    if (alien.isDestroyed()) {
                        destroyed++;
                    }
                    break;
                }
            }
        }
        return destroyed;
    }
}
```

the CollisionManager.java resolve(...) loop, showing bounding-box intersection followed by the interface calls

Special-alien behavior via overridable hooks

Deciding which alien drops a power-up is done without a type test. `Alien` declares `dropsPickup()` returning `false`; `BossAlien` overrides it to return `true`. The game loop asks each alien `dropsPickup()` and treats them all uniformly, so the boss's special behavior lives inside `BossAlien` and adding a new special alien means overriding the hook rather than editing the loop.

```java
public boolean dropsPickup() {
    return false;
}
```

In the Alien class we have this method

```java
@Override
public boolean dropsPickup() {
    return true;
}
```

And in the BossAlien class which is the subclass of the Alien class overrides this method.

```
private void spawnPickupOnBossDeath() {
    for (Alien alien : aliens.getEntities()) {
        if (alien.dropsPickup() && alien.isDestroyed() && !alien.isPickupSpawned()) {
            alien.markPickupSpawned();
            double px = alien.getX() + alien.getWidth() / 2.0;
            double py = alien.getY() + alien.getHeight() / 2.0;
            powerUps.add(makeRandomPickup(px, py));
        }
    }
}
```

And this is how it is called inside the GameScreen class

From the Programmer's Perspective. You override takeHit and activeAnimation() per subtype; the base type defines the call sites.

From the User's Perspective. The loop and the collision code work against supertypes and interfaces, and the JVM resolves the correct implementation at run time.

7. Subtyping & Multityping

Inheritance concerns the implementation (creator programmer) while subtyping concerns the interface (user programmer); the two are independent. A type B is a subtype of A when the interface of A is included in the interface of B. Multiple inheritance is discouraged, but multityping — one class being several interface types at once — is allowed and encouraged, and a class that implements an interface is thereby a subtype of both that interface and `Object`.

How it works

`Alien` is the project's multityping showpiece: a single class that is three interface types at once, implementing `Movable`, `Destructible`, and `Shooter`. The same alien object is seen as a `Movable` by the movement code, as a `Destructible` by the collision code, and as a `Shooter` by the code that drains its buffered shots — each collaborator sees only the facet it needs. A second, non-GUI example is `Projectile`, which implements both `Movable` (it advances itself each frame) and `Damaging` (it deals damage on contact); the same bolt presents its movement facet to the update loop and its damage facet to the collision code.

```
public abstract class Alien extends Entity implements Movable, Destructible, Shooter {
```

```
public abstract class Projectile extends Entity implements Movable, Damaging {
```

the Alien.java header line implements Movable, Destructible, Shooter, and the Projectile.java header line implements Movable, Damaging

Why I did this

Multityping lets one object serve several roles without the conflict problems of multiple inheritance, and it keeps each collaborator decoupled: the collision code cannot accidentally move an alien, because through the `Destructible` view it only sees `takeHit` and `isDestroyed`.

Subtyping shown in client program :

```
Weapon weapon = new StandardWeapon(width, boltFrames);

int shipWidth = 180, shipHeight = 71;
double startX = 90;
double startY = (height - shipHeight) / 2.0;
this.player = new Spaceship(startX, startY, shipWidth, shipHeight,
                            input, playerAnim, speed: 6.0, height,
                            weapon, boltFrames, projectiles, fireRate: 10,
                            PLAYER_HEALTH, width);
```

Here specifically on the last line, the variable weapon is **declared using the general (super) type** Weapon, an abstract class, while the object actually created and assigned to it is of the **more specific (sub) type** StandardWeapon. This is the essence of subtyping: the same object can be legitimately referred to either by its own concrete class (StandardWeapon) or by any of its supertypes (Weapon), and the rest of the program is written to depend only on the general type, without needing to know the specific subtype underneath. This weapon variable is then passed directly into the Spaceship constructor, where it is stored and used for the remainder of the game session.

Multitype demonstration:

```
public abstract class Alien extends Entity implements Movable, Destructible, Shooter {
```

Two of the three interfaces Alien consist:

```
package lastorbit.interfaces;

4 references
public interface Movable {
    3 references
    void moveStep();
}
```

```
2 references
public interface Shooter {
    1 reference
    List<EnemyProjectile> collectPendingShots();
}
```

from gamescreen.java we have

```
aliens.updateAll();
for (Alien alien : aliens.getEntities()) {
    for (EnemyProjectile shot : alien.collectPendingShots()) {
        enemyShots.add(shot);
    }
}
```

This shows the alien being used through one of its multiple types: although the loop variable is declared as Alien, the operation invoked belongs to the Destructible interface that Alien implements, demonstrating that the alien object can be treated through its interface capabilities."

8. Exception Handling

Exceptions in this project are always caught by **specific** type — never with a blanket `catch(Exception e)` — and where several are caught together they are ordered specific-before-general.

Checked vs unchecked, and the hierarchy

Java distinguishes **checked** exceptions (subclasses of `Exception` that the compiler forces a method to either catch or declare with `throws`) from **unchecked** exceptions (subclasses of `RuntimeException`, not enforced by the compiler). This project's custom exceptions are deliberately **checked**: they extend `Exception`, so the compiler guarantees that any code which can fail to load an asset or compose an invalid rule must consciously deal with that possibility. The hierarchy is rooted at `GameException`, with `AssetLoadException` (a sprite or image failed to load) and `InvalidRuleException` (an illegal rule was composed) extending it.

```java
public class GameException extends Exception {

    public GameException(String message) {
        super(message);
    }

    public GameException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

```java
public class InvalidRuleException extends GameException {
    public InvalidRuleException(String message) {
        super(message);
    }
}
```

the exceptions package: GameException.java, AssetLoadException.java, and InvalidRuleException.java

How it works

`ResourceLoader` catches the low-level `IOException` from image loading specifically, wraps it in an `AssetLoadException` that carries the exact file path, and re-throws it — so a missing sprite produces a clear, typed failure rather than a raw stack trace. The high-score reader catches only `IOException`; the music loader catches only the specific audio exceptions it can encounter. Nowhere is a generic `Exception` caught.

```java
public final class ResourceLoader {

    private ResourceLoader() {
    }

    public static BufferedImage loadImage(String path) throws AssetLoadException {
        File file = new File(path);
        if (!file.exists()) {
            throw new AssetLoadException(
                "Asset not found: " + file.getAbsolutePath(), cause: null);
        }
        try {
            BufferedImage image = ImageIO.read(file);
            if (image == null) {
                throw new AssetLoadException("Not a readable image: " + path, cause: null);
            }
            return image;
        } catch (IOException e) {
            throw new AssetLoadException("Failed to read image: " + path, e);
        }
    }
```

the ResourceLoader.java loadImage method, showing the specific catch (IOException e) that wraps and re-throws as AssetLoadException.

From the Programmer's Perspective. You define one checked exception type per failure mode and declare throws where a method can fail, pushing the decision of how to react up to the caller.

From the User's Perspective. Callers catch only the specific exceptions they can handle; a failed asset load surfaces as a readable message identifying the missing file.

9. Rule Customizability & the Open/Closed Principle

The Open/Closed Principle means a module is **open for extension but closed for modification**. The customizable rule system is the project's headline demonstration, and the feature the whole design is built around.

How it works

A `Rule` pairs a `Trigger` (a condition observed each frame) with an `Effect` (what happens when the condition is met). The `RuleEngine` evaluates every rule once per frame and knows nothing about any concrete trigger or effect — it holds only `Trigger` and `Effect` references and calls their interface methods. **There is no `instanceof` and no `switch` on rule type anywhere in the engine.**

To add a brand-new trigger (for example, “the player has not been hit for ten seconds”) or a new effect (“rapid fire”), the creator programmer writes **one new class** and registers it in the `RuleCatalog`. The `RuleEngine` is never modified. That is the Open/Closed Principle realised directly.

```java
public class RuleEngine {

    private final List<Rule> rules;

    public RuleEngine() {
        this.rules = new ArrayList<>();
    }
    public void addRule(Rule rule) throws InvalidRuleException {
        for (Rule existing : rules) {
            if (existing.sameAs(rule)) {
                throw new InvalidRuleException(
                    message: "Duplicate rule: that trigger and effect pair is already set.");
            }
        }
        rules.add(rule);
    }
    public void evaluateAll(GameState state, GameContext context) {
        for (Rule rule : rules) {
            rule.evaluate(state, context);
        }
    }

    public int getRuleCount() {
        return rules.size();
    }

    public List<Rule> getRules() {
        return rules;
    }
}
```

```java
public class LowHealthTrigger implements Trigger {

    private final int threshold;

    public LowHealthTrigger(int threshold) {
        this.threshold = threshold;
    }

    @Override
    public boolean isMet(GameState state) {
        return state.getPlayerHealth() <= threshold;
    }

    @Override
    public String getName() {
        return "Health drops to " + threshold;
    }
}
```

```java
public class ShieldEffect implements Effect {

    private final int hits;

    public ShieldEffect(int hits) {
        this.hits = hits;
    }

    @Override
    public void apply(GameContext context) {
        context.grantShield(hits);
    }

    @Override
    public String getName() {
        return "Gain a shield (" + hits + " hits)";
    }
}
```

a concrete trigger and effect, e.g. LowHealthTrigger.java and ShieldEffect.java, each a small class implementing the interface

Reading and acting on the game safely

The rule system keeps “looking at the game” and “changing the game” strictly separate, using two different objects. Triggers receive a read-only `GameState`: it is a snapshot that exposes **only getters** (such as `getPlayerHealth()`, `getKillStreak()`, `isBossActive()`), so a trigger can observe the game but has no way to alter it. Effects instead receive a `GameContext`, which exposes **only actions** (such as `freezeAllAliens()`, `grantShield()`, `healPlayer()`, `wipeAllAliens()`, and `requestBoss()`), so an effect can change the game but is never handed the raw state to read. In short: a trigger can look but not touch; an effect can touch but not look. This clean split prevents a rule from accidentally creating a feedback loop, and it is why triggers and effects can be combined freely.

```
public int getKillStreak()         { return killStreak; }
public int getPlayerHealth()       { return playerHealth; }
public int getPlayerMaxHealth()    { return playerMaxHealth; }
public boolean isBossActive()      { return bossActive; }
public boolean wasLastWaveClean()  { return lastWaveWasClean; }
public int getAliensOnScreen()     { return aliensOnScreen; }
public double getNearestAlienX()   { return nearestAlienX; }
public int getScreenWidth()        { return screenWidth; }
```

the GameState.java class (its getters)

```
public void healPlayer(int amount) {
    player.heal(amount);
}
public void grantShield(int hits) {
    player.attachShield(new Shield(hits));
}
public void unfreezeAllAliens() {
    for (Alien a : aliens.getEntities()) {
        a.setFrozen(frozen: false);
    }
}
```

the GameContext.java class action methods

A second Open/Closed example — adaptive difficulty

The same principle appears in wave generation. `WaveManager` asks a `DifficultyController` for the next wave's `WaveParameters`. Adaptive difficulty was added by plugging the controller into that existing seam — `Wave`, `WaveParameters`, and the enemy factory did not change at all.

From the Programmer's Perspective. You extend the game by adding classes, not by editing the engine. Abstraction is what makes this possible: the engine depends only on the Trigger and Effect types.

From the User's Perspective. The player exercises extensibility directly, composing up to three custom rules before the game starts, and the engine runs whatever they build.

10. File Input/Output — High Scores

High scores are persisted to a plain-text file using the standard `java.io` package, with no database. `HighScoreStore` is the only class that touches the file, so the storage format lives in exactly one place. Each entry holds a name, a score, and the wave reached, written one per line as comma-separated values; the store loads the table on startup, decides whether a new score qualifies for the top five, inserts and trims, and writes back.

File streams are opened with **try-with-resources**, so they close automatically even if an error occurs, and a corrupt or missing file is handled gracefully — a bad line is skipped and a missing file simply starts an empty table, rather than crashing the game.

```
public class HighScoreStore {

    private static final int MAX_ENTRIES = 5;
    private final String filePath;
    private final List<ScoreEntry> entries;

    public HighScoreStore(String filePath) {
        this.filePath = filePath;
        this.entries = new ArrayList<>();
        load();
    }
    private void load() {
```

```
private void load() {
    entries.clear();
    File file = new File(filePath);
    if (!file.exists()) {
        return;
    }
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            if (line.trim().isEmpty()) {
                continue;
            }
            ScoreEntry entry = ScoreEntry.fromLine(line);
            if (entry != null) {
                entries.add(entry);
            }
        }
    } catch (IOException e) {
    }
    sortAndTrim();
}
```

```java
private void save() {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
        for (ScoreEntry entry : entries) {
            writer.write(entry.toLine());
            writer.newLine();
        }
    } catch (IOException e) {
    }
}
```

the HighScoreStore.java load() and save() methods,

```
RO,39,5
1,32,4
ERMI,18,3
AF,8,2
RO,7,1
```

the generated highscores.txt

11. Sound

Background music is played through the standard `javax.sound.sampled` package, so no external audio library is needed. A single `SoundManager` class owns all audio: it loads a `.wav` file, opens it as a `Clip`, and loops it continuously while the game runs. Keeping every audio concern inside this one class is information hiding again — the rest of the game never deals with audio details, it only asks the manager to play the music.

The loader is written to fail silently: if the music file is missing or is not a format Java can decode, the specific exceptions (`UnsupportedAudioFileException`, `IOException`, `LineUnavailableException`) are each caught individually and the game simply runs without sound rather than crashing. This is a deliberate robustness choice — audio is non-essential, so a problem with it should never stop the game from being playable.

```java
public class SoundManager {

    private Clip musicClip;


    public void playMusicLoop(String path) {
        try {
            File file = new File(path);
            if (!file.exists()) {
                System.out.println("MUSIC: loading " + file.getAbsolutePath());
                return;
            }
            AudioInputStream stream = AudioSystem.getAudioInputStream(file);
            musicClip = AudioSystem.getClip();
            musicClip.open(stream);
            musicClip.loop(Clip.LOOP_CONTINUOUSLY);
            musicClip.start();
        } catch (UnsupportedAudioFileException e) {
            System.out.println("MUSIC: unsupported format (is it really a WAV? MP3 won't work): " + e.getMessage());
        } catch (IOException e) {
            System.out.println("MUSIC: couldn't read file: " + e.getMessage());
        } catch (LineUnavailableException e) {
            System.out.println("MUSIC: no audio line available: " + e.getMessage());
        }
    }
```

12. Concurrency — the Game Loop Thread

The game runs on its own thread so that updating and rendering happen at a steady frame rate independently of the rest of the program. `GameLoop` is written to **implement `Runnable`** rather than to extend `Thread`: implementing the interface leaves the class free to extend something else if needed and cleanly separates the task (what to run) from the worker (the thread that runs it). The loop repeatedly updates the active screen and requests a repaint, then sleeps for the remainder of the frame to hold a fixed rate.

```
public class GameLoop implements Runnable {

    private final GamePanel panel;
    private final int targetFps;
    private volatile boolean running;
    private Thread thread;
```

Volatile meaning it is safely seen across threads

```
@Override
public void run() {
    final long frameTimeNanos = 1_000_000_000L / targetFps;
    while (running) {
        long frameStart = System.nanoTime();

        panel.updateGame();
        panel.repaint();

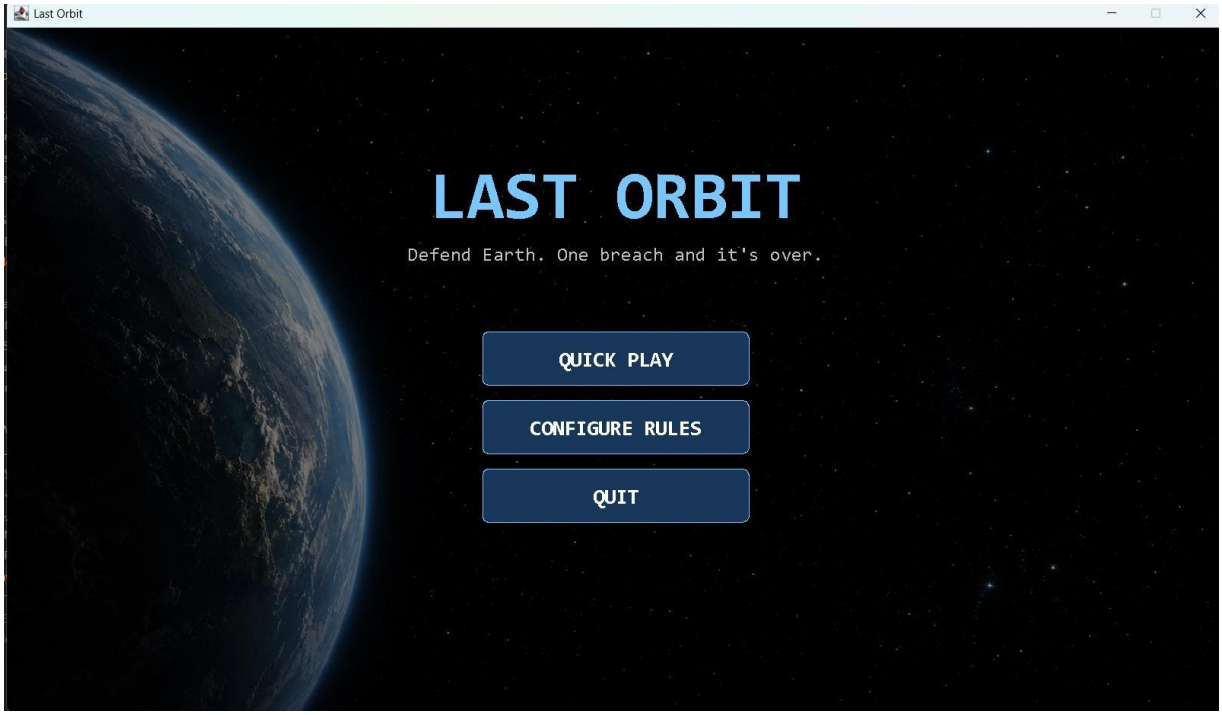
        long elapsed = System.nanoTime() - frameStart;
        long sleepNanos = frameTimeNanos - elapsed;
        if (sleepNanos > 0) {
            try {
                Thread.sleep(sleepNanos / 1_000_000L,
                             (int) (sleepNanos % 1_000_000L));
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                running = false;
            }
        }
    }
}
```

the GameLoop.java class header implements Runnable, the run() method with the update/repaint/sleep cycle, and start() creating the thread

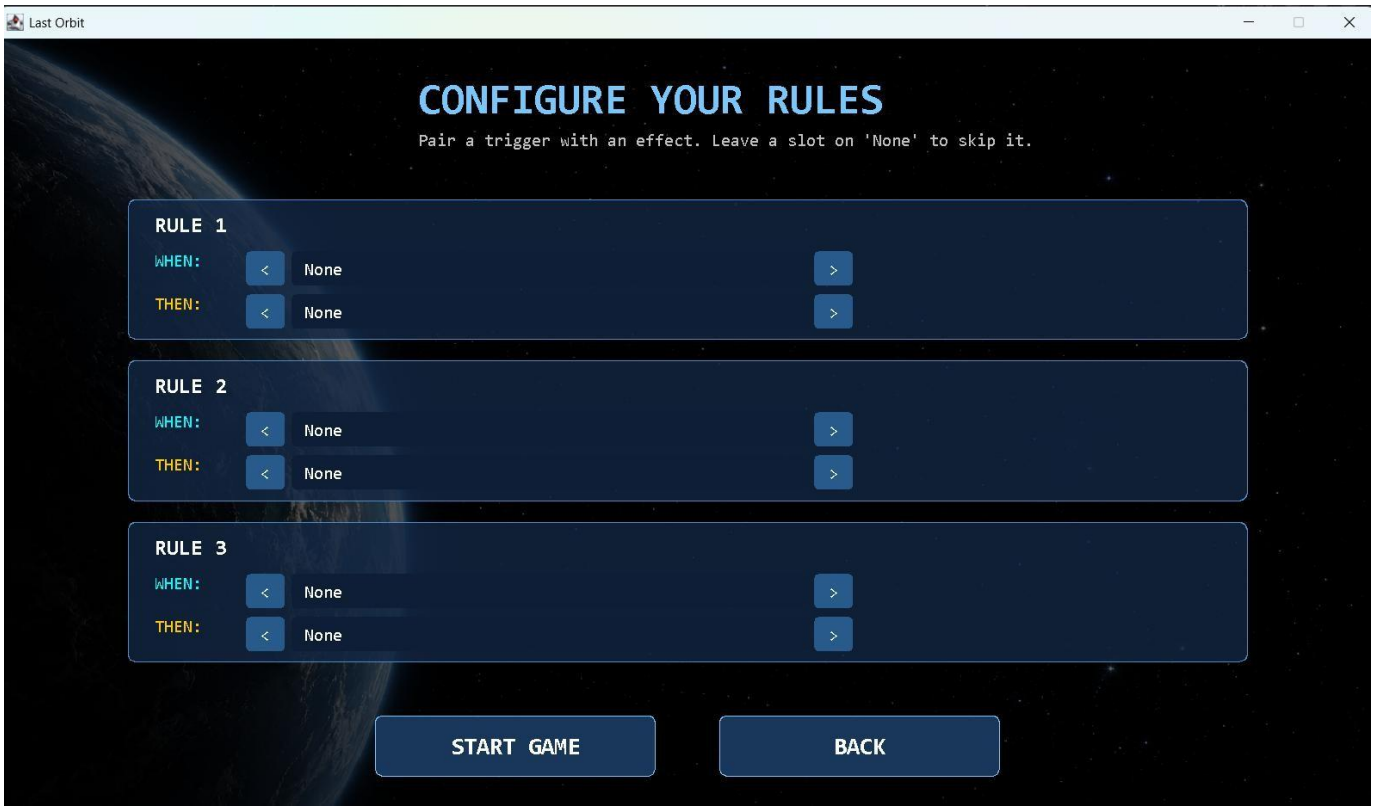
12. Graphical User Interface

The interface is built entirely with Java Swing and `Graphics2D`. A small `Screen` abstraction represents one screen of the application — the main menu, the rule-configuration screen, and the game itself are each a `Screen`. A `ScreenManager` holds whichever screen is active and forwards updating, drawing, and input to it, so switching screens is simply replacing the active reference, with no conditional logic deciding "which screen are we on." Adding a new screen is a new `Screen` subclass that the manager never has to change for — the same Open/Closed idea applied to the interface layer.

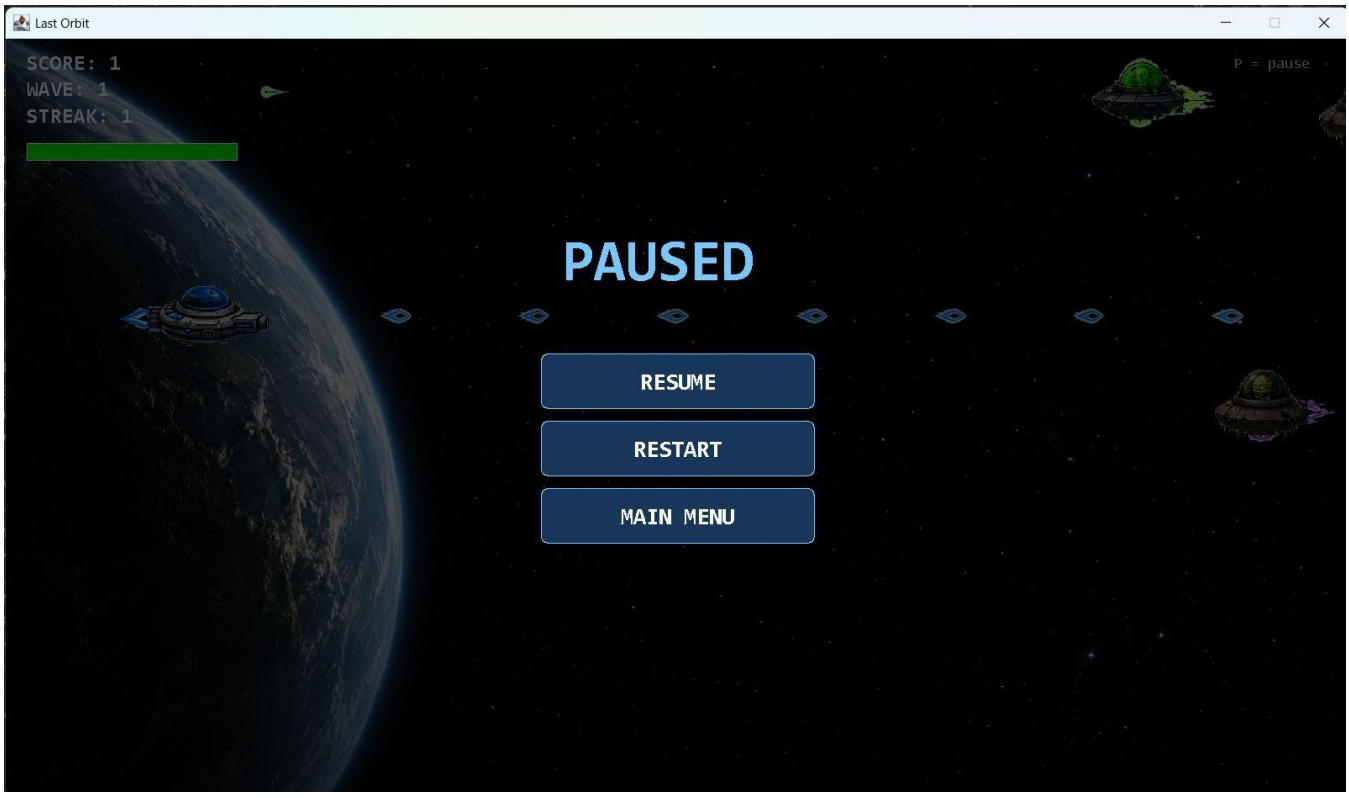
Pause and game-over are handled as overlay states drawn on top of the live game, so the game state is preserved underneath and the player can resume exactly where they left off. Menus use lightweight clickable `Button` helpers, and the rule-configuration screen lets the player step through the available triggers and effects and adjust their values before starting.

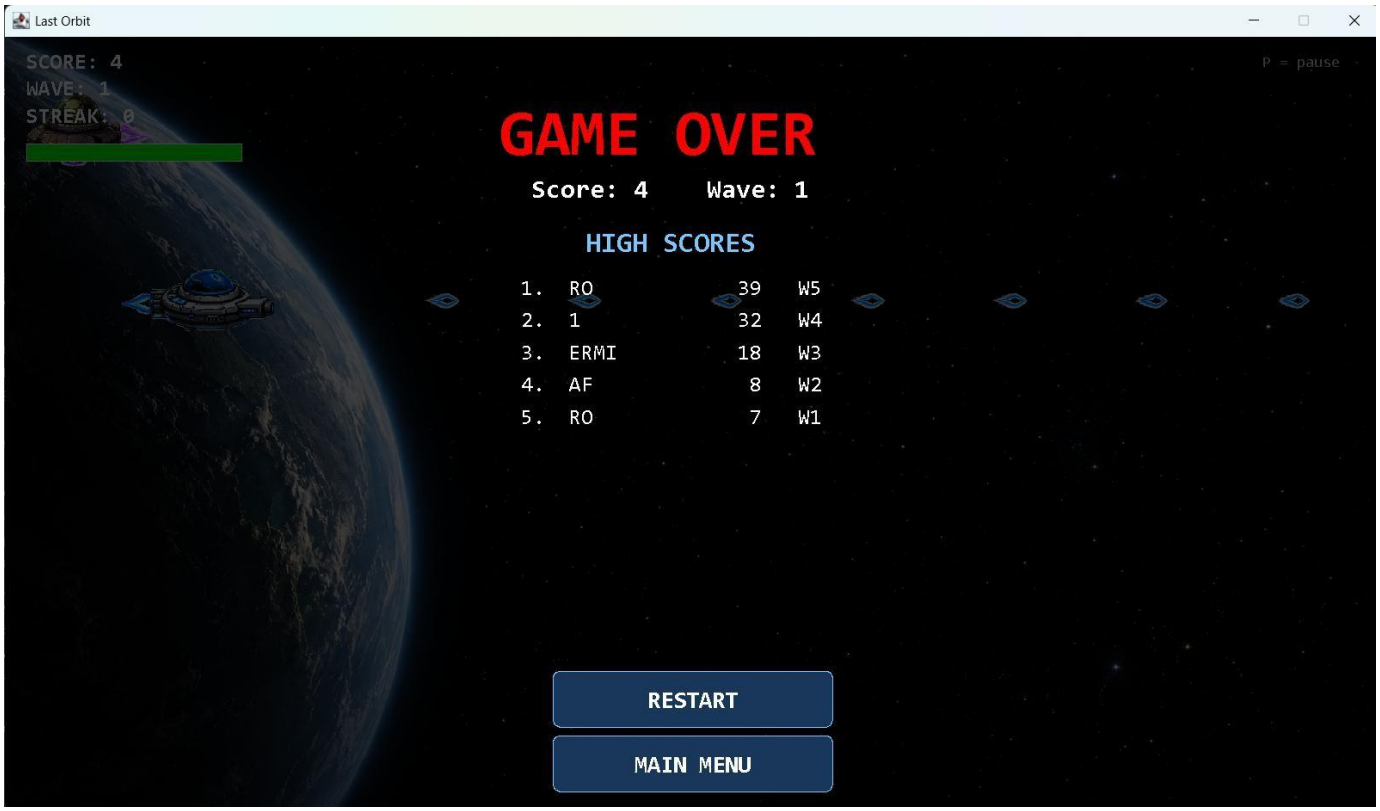


The Main Menu screen in-game.



The Rule Configuration screen in-game, showing the WHEN(trigger)/THEN(effect) slots





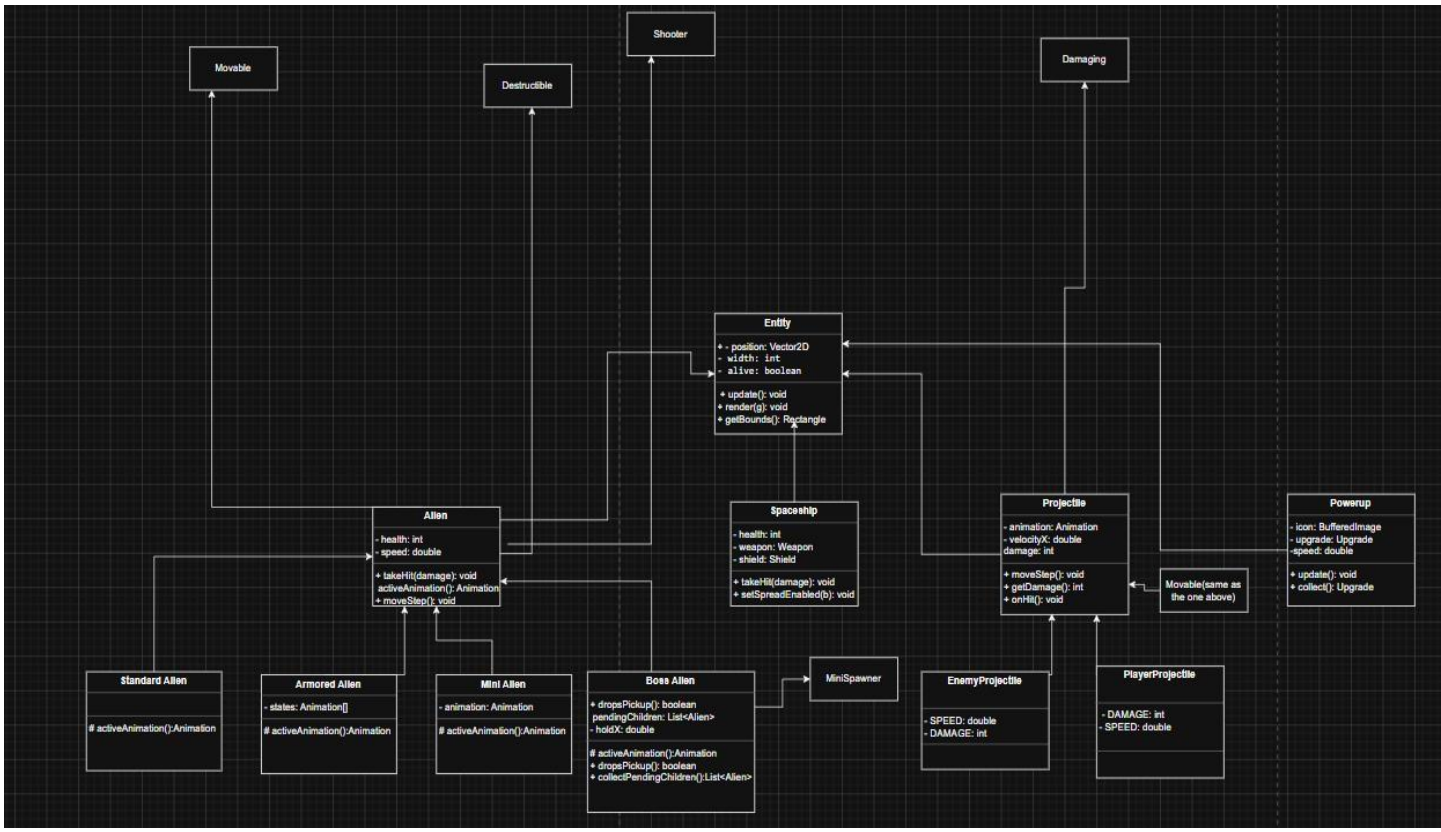
The Pause and Game Over overlay in-game.

Class Diagram

Two class diagrams are included, one per cluster, to keep each readable.

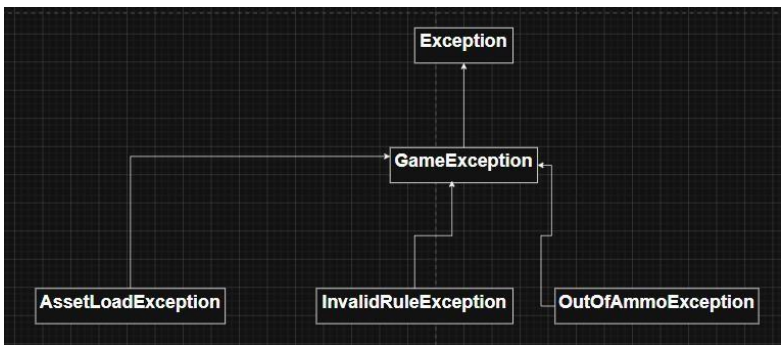
Entity cluster

The first diagram shows the main game-object hierarchy. The abstract `Entity` is the root; `Spaceship`, `Alien`, `Projectile`, and `PowerUp` extend it (solid inheritance arrows). `Alien` is itself an abstract superclass specialised by `StandardAlien`, `ArmoredAlien`, `MiniAlien`, and `BossAlien`, and `Projectile` by `PlayerProjectile` and `EnemyProjectile`. The dashed realization arrows show multityping: `Alien` realises `Movable`, `Destructible`, and `Shooter`; `Projectile` realises `Movable` and `Damaging`; and `BossAlien` additionally realises `MiniSpawner`. This diagram is where inheritance, abstraction, and multityping are all visible at once.



Exception cluster

The second diagram shows the custom exception hierarchy. `GameException` is the base class and extends Java's built-in `Exception`, which makes every custom exception a **checked** exception — the compiler forces callers to handle or declare it. `AssetLoadException` and `InvalidRuleException` extend `GameException` (solid inheritance arrows). Using checked exceptions here is a deliberate choice: loading an asset or composing a rule can genuinely fail, and the compiler should make that failure impossible to ignore.



the exception cluster: Exception → GameException → AssetLoadException, InvalidRuleException.

User Guide

- **Move:** Up/Down arrow keys (or W/S). The ship is fixed at the left and moves vertically.
- **Fire:** automatic while a wave is active.
- **Pause:** P or Esc — Resume, Restart, or Main Menu.
- **Goal:** destroy aliens and survive waves; a single alien reaching Earth ends the game. Clearing a wave restores some health.
- **Boss waves:** every fifth wave. Defeating the boss releases a power-up that drifts left — line up vertically to catch it.
- **Custom rules:** choose *Configure Rules* from the menu to pair conditions (WHEN) with effects (THEN) before playing.
- **High scores:** a top-five score prompts for your name and is saved to disk.

Conclusion

Last Orbit demonstrates every OOP concept from the course in working code: information hiding with its three engineering steps, encapsulation, inheritance as white-box specialization, abstraction through abstract classes and single-method interfaces, composition as black-box reuse with runtime component swapping, all four kinds of polymorphism with **no-RTTI dynamic** dispatch throughout, subtyping and multityping, a specific-catch checked-exception hierarchy, and the Open/Closed Principle realised twice — in the player-facing rule engine and in adaptive difficulty. It also covers file I/O for persistence and a threaded game loop.